

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: Расстояние Левенштейна
Вариант: 14а

Студент гр. 4343

Гизатов Я.Р.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание 1

Над строкой ε (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. $Replace(\varepsilon, a, b)$ – заменить символ a на символ b .
2. $insert(\varepsilon, a)$ – вставить в строку символ a (на любую позицию).
3. $delete(\varepsilon, b)$ – удалить из строки символ b .

Каждая операция может иметь некоторую цену выполнения (положительное число).

Даны две строки A и B , а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки A в строку B .

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – AA ; третья строка – B .

Выходные данные: одно число – минимальная стоимость операций.

Sample Input:

1 1 1

entrance

reenterable

Sample Output:

5

Задание 2

Над строкой ε (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. $replace(\varepsilon, a, b)$ – заменить символ a на символ b .
2. $insert(\varepsilon, a)$ – вставить в строку символ a (на любую позицию).
3. $delete(\varepsilon, b)$ – удалить из строки символ b .

Каждая операция может иметь некоторую цену выполнения (положительное число).

Даны две строки A и B, а также три числа, отвечающие за цену каждой операции. Определите последовательность операций (редакционное предписание) с минимальной стоимостью, которые необходимы для превращения строки A в строку B.

Пример (все операции стоят одинаково)

M	M	M	R	I	M	R	R
C	O	N	N		E	C	T
C	O	N	E	H	E	A	D

Пример (цена замены 3, остальные операции по 1)

M	M	M	D	M	I	I	I	I	D	D
C	O	N	N	E					C	T
C	O	N		E	H	E	A	D		

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – A; третья строка – B.

Выходные данные: первая строка – последовательность операций (M – совпадение, ничего делать не надо; R – заменить символ на другой; I – вставить символ на текущую позицию; D – удалить символ из строки); вторая строка – исходная строка A; третья строка – исходная строка B.

Sample Input:

1 1 1

entrance

reenterable

Sample Output:

IMIMMIMMRRM

entrance

reenterable

Задание 3

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: `pedestal` → `stal`.
- Затем необходимо заменить два последних символа: `stal` → `stie`.
- Потом нужно добавить символ в конец строки: `stie` → `stien`.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. ($S, 1 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. ($T, 1 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L , равное расстоянию Левенштейна между строками S и T .

Sample Input:

`pedestal`

`stien`

Sample Output:

`7`

Задание 4 (индивидуализация)

Вар. 14. Методом динамического программирования вычислить:

14а. длину наибольшей общей подпоследовательности двух строк.

Задание 1

Выполнение работы

Для выполнения задания был реализован алгоритм на C++ вычисления редакционного расстояния (расстояния Левенштейна) с возможностью задавать различные стоимости операций замены, вставки и удаления, после чего добавили в эту реализацию подробное логирование всех промежуточных вычислений в файл, а в терминал вывели только итоговое значение.

Основные структуры данных

std::vector<std::vector<int>> dp Двумерный вектор размером $(n + 1) \times (m + 1)$ типа `int` для хранения минимальных стоимостей преобразования префиксов строк, где `dp[i][j]` хранит стоимость преобразования первых i символов строки A в первые j символов строки B.

Описание алгоритма

Работа алгоритма начинается с инициализации таблицы `dp`, где нулевой столбец заполняется значениями i умноженными на стоимость удаления, что соответствует удалению всех символов из префикса строки A, а нулевая строка заполняется значениями j умноженными на стоимость вставки, что соответствует вставке всех символов в пустую строку для получения префикса строки B. Затем программа последовательно перебирает все пары индексов i от 1 до n и j от 1 до m , где n и m это длины строк A и B соответственно. На каждом шаге сравниваются символы `A[i-1]` и `B[j-1]`, и если они равны, то значение `dp[i][j]` копируется из диагональной ячейки `dp[i-1][j-1]`, что означает отсутствие необходимости в операции замены. Если символы не равны, то вычисляются три возможные стоимости операции: замена символа A на B через `dp[i-1][j-1]` плюс стоимость замены, вставка символа B через `dp[i][j-1]` плюс стоимость вставки и удаление символа A через `dp[i-1][j]` плюс стоимость удаления, после чего выбирается минимальное значение и записывается в `dp[i][j]`. Параллельно с заполнением таблицы все промежуточные вычисления записываются в лог-файл: выводится текущая пара сравниваемых символов, значения трёх вариантов и выбранный минимум, а после завершения обработки каждой строки

таблица выводится целиком. После заполнения всей таблицы значение $dp[n][m]$ представляет собой минимальную стоимость преобразования всей строки A в строку B, которое выводится в терминал, а подробный ход вычислений остаётся в файле log1.txt.

void print_table (std::ofstream& log, const std::vector<std::vector<int>> & dp, const std::string& A, const std::string& B) которая отвечает за вывод текущего состояния таблицы динамического программирования в лог-файл. Эта функция принимает четыре параметра: ссылку на объект `std::ofstream log`, куда производится запись, константную ссылку на двумерный вектор `dp` с целочисленными значениями, а также константные ссылки на строки A и B. Внутри функция сначала определяет длины n и m строк A и B, затем выводит в лог отступ из двух пробелов и через пробел перечисляет все символы строки B, формируя заголовок таблицы. После этого функция в цикле проходит по всем i от 0 до n , причём если i равно нулю, перед выводом строки ставится пробел, а если i больше нуля, выводится соответствующий символ из строки A. Для каждой строки i функция выводит все значения $dp[i][j]$ для j от 0 до m , используя манипулятор `std::setw(3)` для выравнивания каждого числа по ширине в три символа, после чего переходит на новую строку. В конце вывода таблицы функция добавляет пустую строку для визуального разделения блоков в лог-файле.

main основная функция не возвращает никакого значения и выполняет все остальные действия: считывание входных данных, инициализацию таблицы, заполнение её по алгоритму Вагнера-Фишера с записью промежуточных вычислений в лог через вызовы `print_table`, а также вывод финального результата в консоль.

Оценка сложности

Временная сложность алгоритма составляет $O(n \times m)$, где n – длина строки A, m – длина строки B, поскольку программа выполняет двойной вложенный цикл, перебирающий все возможные пары префиксов строк, и

внутри каждой итерации выполняется константное количество операций (сравнение символов, сложение и нахождение минимума из трёх чисел).

Оценка по памяти

Пространственная сложность также составляет $O(n \times m)$, так как используется двумерный массив `dp` размером $(n + 1) \times (m + 1)$, хранящий целочисленные значения для всех подзадач. При этом дополнительная память тратится на хранение двух строк `A` и `B`, а также на буферизацию вывода в файл, но эти затраты асимптотически меньше, чем $O(n \times m)$. Алгоритм не может быть выполнен быстрее, чем за $O(n \times m)$ в худшем случае, поскольку необходимо вычислить расстояния для всех комбинаций префиксов, а используемая таблица полностью сохраняет все промежуточные результаты, что даёт оптимальную асимптотику для данного подхода.

Задание 2

Выполнение работы

Для выполнения задания был реализован алгоритм на C++ вычисления редакционного расстояния с возможностью задавать стоимости операций замены, вставки и удаления, а также с восстановлением последовательности операций редактирования. В программу добавлено подробное логирование всех промежуточных вычислений в файл "log2.txt" под управлением флага `log_flag`: при его включении записываются начальная таблица, результаты каждого сравнения символов, состояние таблицы после каждой строки и финальное расстояние. В терминал выводятся последовательность операций (M – совпадение, R – замена, I – вставка, D – удаление), исходная строка A и целевая строка B.

Основные структуры данных

`std::vector<std::vector<int>>` `dp` Двумерный вектор размером $(n + 1) \times (m + 1)$ типа `int` для хранения минимальных стоимостей преобразования префиксов строк, где `dp[i][j]` хранит стоимость преобразования первых i символов строки A в первые j символов строки B.

Описание алгоритма

В программе реализован алгоритм Вагнера – Фишера для вычисления минимальной стоимости преобразования одной строки в другую с учётом заданных стоимостей операций замены, вставки и удаления. Сначала инициализируется двумерная таблица `dp` размером $(n+1) \times (m+1)$, где n – длина строки A, m – длина строки B. Первый столбец таблицы заполняется значениями $i * \text{costD}$, что соответствует стоимости удаления i символов из строки A. Первая строка заполняется значениями $j * \text{costI}$, что соответствует стоимости вставки j символов в пустую строку для получения B. Затем для каждой ячейки (i, j) начиная с 1 до n и 1 до m выполняется переход: если символы $A[i-1]$ и $B[j-1]$ равны, то $dp[i][j] = dp[i-1][j-1]$ без добавления стоимости; если символы различны, то $dp[i][j] = \min(dp[i-1][j-1] + \text{costR}, dp[i][j-1] + \text{costI}, dp[i-1][j] + \text{costD})$, то есть выбирается минимальная

стоимость среди замены символа, вставки символа в A или удаления символа из A. После заполнения всей таблицы выполняется обратный проход для восстановления последовательности операций: начиная с ячейки (n, m), программа проверяет, какой из вариантов привёл к текущему значению – если символы равны и значение совпадает с диагональю, добавляется операция 'M' (match); иначе если значение совпадает с диагональю плюс стоимость замены, добавляется 'R' (replace); если совпадает со значением слева плюс стоимость вставки, добавляется 'I' (insert); если совпадает со значением сверху плюс стоимость удаления, добавляется 'D' (delete). После завершения обратного прохода последовательность операций разворачивается и выводится в консоль вместе с исходными строками. Дополнительно реализовано логирование каждого шага в файл "log2.txt" при установленном флаге log_flag.

print_table(std::ofstream& log, const std::vector<std::vector<int>>& dp, const std::string& A, const std::string& B) – принимает на вход ссылку на объект std::ofstream для записи в лог-файл, константную ссылку на таблицу dp, а также константные ссылки на строки A и B. Функция выводит в лог отформатированное представление текущего состояния таблицы dp: сначала печатается строка с заголовками столбцов (символы строки B), затем для каждой строки i таблицы выводится символ A[i-1] (или пробел для нулевой строки), после чего выводятся все значения dp[i][j] с шириной поля 3 символа для выравнивания. В конце каждого вызова функция добавляет пустую строку для разделения таблиц в лог.

main – основная функция программы, которая управляет всем процессом: считывает входные данные, инициализирует таблицу dp, выполняет заполнение таблицы по алгоритму Вагнера-Фишера с условным логированием в файл "log2.txt" (при установленном флаге log_flag), восстанавливает последовательность операций редактирования обратным проходом по таблице, разворачивает полученную строку операций и выводит в консоль последовательность операций, исходную строку A и

целевую строку В. Также функция управляет открытием и закрытием лог-файла.

Оценка сложности

Реализованный алгоритм Вагнера–Фишера имеет временную сложность $O(n \times m)$, где $n = |A|$, $m = |B|$. Это обосновывается тем, что программа заполняет таблицу `dp` размером $(n + 1) \times (m + 1)$ с помощью двух вложенных циклов: внешний цикл по i от 1 до n и внутренний по j от 1 до m . Внутри каждой итерации выполняется константное количество операций – сравнение символов, вычисление минимума из трёх значений (замена, вставка, удаление) и присваивание. Таким образом, количество итераций равно $n \cdot m$, и каждая требует $O(1)$ времени. Дополнительные этапы: инициализация первого столбца и первой строки занимают $O(n)$ и $O(m)$ соответственно, что пренебрежимо мало по сравнению с $n \cdot m$. Обратный проход для восстановления последовательности операций выполняется за $O(n + m)$, так как на каждом шаге происходит переход к соседней ячейке (уменьшается либо i , либо j , либо оба). Логирование в файл добавляет константные затраты на каждый шаг (вывод строк в поток), но не изменяет асимптотическую оценку, поскольку объём выводимых данных пропорционален количеству итераций (каждая итерация может породить несколько строк лога). Итоговая временная сложность – $O(n \times m)$.

Оценка по памяти

Программа использует $O(n \times m)$ дополнительной памяти. Основной объём занимает двумерный вектор `dp` размером $(n + 1) \times (m + 1)$, каждый элемент которого – целое число (`int`). При n и m порядка нескольких тысяч это может быть существенно. Также хранятся исходные строки A и B ($O(n + m)$) и строка операций `ops` ($O(n + m)$ в худшем случае). Флаг логирования и файловый поток не учитываются в асимптотической оценке, так как их размер не зависит от входных данных. Таким образом, сложность по памяти составляет $O(n \cdot m)$.

Задание 3

Выполнение работы

Для выполнения задания был реализован алгоритм на C++ вычисления редакционного расстояния (расстояния Левенштейна), после чего добавили в эту реализацию подробное логирование всех промежуточных вычислений в файл, а в терминал вывели только итоговое значение.

Основные структуры данных

`std::vector<std::vector<int>>` **dp** Двумерный вектор размером $(n + 1) \times (m + 1)$ типа `int` для хранения минимальных стоимостей преобразования префиксов строк, где `dp[i][j]` хранит стоимость преобразования первых i символов строки s в первые j символов строки t .

Описание алгоритма

Алгоритм, реализованный в программе, предназначен для вычисления расстояния Левенштейна между двумя строками, которое представляет собой минимальное количество операций редактирования, необходимых для преобразования одной строки в другую. В данном случае разрешены три типа операций: замена символа, вставка символа и удаление символа, каждая из которых имеет стоимость, равную единице. Для решения задачи используется метод динамического программирования, а именно алгоритм Вагнера–Фишера. Суть метода заключается в заполнении двумерной таблицы `dp` размером $(n+1)$ на $(m+1)$, где n – длина первой строки s , m – длина второй строки t . Элемент `dp[i][j]` таблицы хранит минимальную стоимость преобразования префикса строки s длины i в префикс строки t длины j . На первом этапе происходит инициализация границ таблицы: `dp[0][j]` для всех j от 0 до m заполняется значениями j , что соответствует стоимости вставки j символов для получения строки t из пустой строки; аналогично `dp[i][0]` для всех i от 0 до n заполняется значениями i , что соответствует стоимости удаления i символов для превращения префикса s в пустую строку. После инициализации начинается последовательное

заполнение оставшихся ячеек таблицы по строкам. Для каждой пары индексов i от 1 до n и j от 1 до m программа сравнивает символы $s[i-1]$ и $t[j-1]$. Если символы равны, то стоимость $dp[i][j]$ принимается равной $dp[i-1][j-1]$, поскольку дополнительная операция не требуется. Если символы различаются, то вычисляются три возможные стоимости: замена символа ($dp[i-1][j-1] + 1$), вставка символа в первую строку ($dp[i][j-1] + 1$) и удаление символа из первой строки ($dp[i-1][j] + 1$). Из этих трёх значений выбирается минимальное, которое и записывается в $dp[i][j]$. Таким образом, каждая ячейка таблицы заполняется на основе уже вычисленных соседних ячеек, что соответствует оптимальной подструктуре задачи. После того как вся таблица заполнена, искомое расстояние Левенштейна оказывается в ячейке $dp[n][m]$. Помимо основного вычисления, в программе реализовано подробное логирование в файл. При включённом флаге `log_flag` создаётся файл `log3.txt`, в который выводится начальная таблица, затем для каждого сравнения символов записывается, были ли они равны или нет, и какое значение выбрано. После обработки каждой строки исходной строки s в лог выводится текущее состояние всей таблицы. В конце в лог записывается финальное расстояние. Логирование помогает наглядно проследить все этапы работы алгоритма, что полезно для отладки или обучения. После завершения расчётов программа выводит на экран только само расстояние Левенштейна.

Оценка сложности

Временная сложность алгоритма составляет $O(n \times m)$, где n – длина первой строки s , а m – длина второй строки t . Это обосновано тем, что в программе реализованы два вложенных цикла: внешний цикл выполняется n раз (по всем символам строки s), а внутренний цикл выполняется m раз (по всем символам строки t). На каждой итерации внутреннего цикла выполняются только константные операции: сравнение двух символов, вычисление минимума из трёх чисел (либо просто присваивание) и, при включённом логировании, запись в файл. Поскольку количество итераций

равно $n \times m$, а каждая итерация обрабатывается за $O(1)$, общая временная сложность составляет $O(n \times m)$.

Оценка по памяти

Пространственная сложность алгоритма оценивается как $O(n \times m)$ по той же причине: в программе используется двумерный массив `dp` размером $(n + 1) \times (m + 1)$ для хранения всех промежуточных значений динамического программирования. Каждый элемент массива хранит целое число, поэтому объём занимаемой памяти прямо пропорционален произведению длин входных строк. Дополнительная память, необходимая для хранения самих строк `s` и `t`, а также для служебных переменных и буферов логирования, пренебрежимо мала по сравнению с размером таблицы `dp`. Таким образом, основным фактором, определяющим потребление памяти, является именно таблица размером $O(n \times m)$.

Задание 4

Выполнение работы

В рамках выполнения задания по динамическому программированию сделана программа на C++ для решения задания «вычислить длину наибольшей общей подпоследовательности двух строк» с выводом промежуточных вычислений в лог-файл и кратким результатом на экран.

Основные структуры данных

vector<vector<int>> dp – двумерный вектор (таблица) размером $(m+1) \times (n+1)$, где m и n – длины строк. Хранит длины наибольших общих подпоследовательностей для всех префиксов строк. Является основной структурой.

Описание алгоритма

Сначала программа считывает две строки из стандартного ввода. Затем инициализирует двумерную таблицу **dp** размером $(m+1) \times (n+1)$ нулями и открывает лог-файл. Далее двойным циклом заполняет таблицу: для каждой пары символов если они равны, то $dp[i][j] = dp[i-1][j-1] + 1$, иначе $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$, и все сравнения и присваивания записываются в лог. После заполнения таблицы выполняется обратный ход из ячейки $dp[m][n]$ для восстановления самой подпоследовательности: при равенстве символов добавляем символ в строку **lcs** и идём по диагонали, иначе двигаемся вверх или влево в зависимости от того, где значение **dp** больше. Восстановленная строка переворачивается, так как сбор шёл с конца. Затем в лог записывается итоговая подпоследовательность и её длина, а на экран выводится только строка **lcs** и через перевод строки её длина. Лог-файл закрывается, программа завершается.

log_to_file(const string& msg) принимает строковое сообщение и, если глобальный флаг **logOpened** установлен в **true** (что происходит после успешного открытия файла), записывает это сообщение в глобальный выходной файловый поток **logFile**, который связан с файлом **lcs_log.txt**. Эта функция используется для вывода всех промежуточных вычислений, таких

как сравнения символов, значения ячеек таблицы, шаги восстановления подпоследовательности, а также итогового результата, но только в лог-файл, не на экран.

print_dp_matrix(const vector<vector<int>>& dp, const string& s1, const string& s2) принимает заполненную или частично заполненную таблицу **dp** и две исходные строки. Она формирует и выводит в лог-файл через **log_to_file** визуальное представление матрицы: сначала строку с заголовками столбцов (символы второй строки), затем строку с индексами столбцов, а потом для каждой строки таблицы выводит либо индекс строки с символом первой строки (для ненулевых строк), либо просто индекс для нулевой строки, а затем все значения **dp[i][j]** в текущей строке. Это позволяет наглядно отслеживать состояние таблицы динамического программирования после каждого шага заполнения.

main ввод строк, инициализация таблицы, двойной цикл заполнения **dp** по правилам динамического программирования, цикл обратного хода для восстановления подпоследовательности, а также вывод на экран итогового результата и закрытие лог-файла.

Оценка сложности

Сложность алгоритма составляет $O(m \times n)$, где m и n – длины входных строк. Это обосновано тем, что программа заполняет таблицу **dp** размером $(m + 1) \times (n + 1)$ с помощью двух вложенных циклов, каждый из которых выполняется m и n раз соответственно. Внутри каждого шага выполняются константные операции: сравнение символов, вычисление максимума, запись в лог. Таким образом, общее количество итераций пропорционально произведению длин строк.

Оценка по памяти

Сложность составляет $O(m \times n)$. Программа хранит всю таблицу **dp** как двумерный вектор размера $(m+1) \times (n+1)$, каждый элемент которого – целое число. Дополнительно используются строки **s1**, **s2**, **lcs** и несколько целочисленных переменных, но они не влияют на асимптотику. Память под

лог-файл не учитывается в оценке сложности алгоритма, так как это внешнее хранилище.

Тестирование

Задание 1

Таблица 1 – Тестирование задания 1

№ теста	Входные данные	Выходные данные
Тест 1: пустые строки	1 1 1	0
Тест 2: одна строка пустая, вторая непустая	1 1 1 abc	3
Тест 3: строки полностью совпадают	1 1 1 abc abc	0
Тест 4: разные стоимости операций с пустой строкой	5 2 10 abcd	40
Тест 5: строки разной длины без общих символов входные данные:	1 1 1 abc def	3

Задание 2

Таблица 2 – Тестирование задания 2

№ теста	Входные данные	Выходные данные
Тест 1. Обе строки пустые	1 1 1	
Тест 2. Одна строка пустая, другая непустая (стоимость вставки 2, удаления 5)	5 2 10 abc	DDD abc
Тест 3. Одна строка непустая, другая пустая (стоимость удаления 3)	1 1 3 xyz	DDD xyz
Тест 4. Строки одинаковые	2 2 2 hello hello	MMMMM hello hello
Тест 5. Полная замена всех символов с разными стоимостями (замена дешево, вставка и удаление дорого)	1 100 100 abc xyz	RRR abc xyz

Задание 3

Таблица 3 – Тестирование задания 3

№ теста	Входные данные	Выходные данные
Тест 1: Обе строки пустые		0
Тест 2: Первая строка пустая, вторая непустая	abc	3
Тест 3: Вторая строка пустая, первая непустая	xyz	3
Тест 4: Строки полностью совпадают	hello hello	0
Тест 5: Строки не совпадают ни в одной позиции (максимальное расстояние)	aaaa bbbb	4

Задание 4

Таблица 4 – Тестирование задания 4

№ теста	Входные данные	Выходные данные
Тест 1: Пустые строки		0
Тест 2: Одна пустая строка, вторая непустая	abc	0
Тест 3: Одна строка длиной 1, вторая длиной 1, символы совпадают	a a	a 1
Тест 4: Одна строка длиной 1, вторая длиной 1, символы не совпадают	a b	0
Тест 5: Одинаковые строки из повторяющегося символа (проверка счёта)	aaaaa aaaaa	aaaaa 5

Программы для всех случаев отработала корректно.

Вывод

В ходе выполнения лабораторной работы были реализованы четыре алгоритма динамического программирования для решения задач редактирования строк. В первом задании реализован алгоритм Вагнера–Фишера для вычисления минимальной стоимости преобразования строки А в строку В с учётом различных стоимостей операций замены, вставки и удаления. Временная сложность алгоритма составляет $O(n \times t)$, по памяти – $O(n \times t)$. Во втором задании дополнена функциональность восстановлением редакционного предписания с помощью обратного прохода по таблице dp, что позволило выводить последовательность операций (M, R, I, D). Третье задание реализует классическое расстояние Левенштейна, где все операции имеют стоимость 1. В четвёртом задании (индивидуальный вариант 14а) вычислена длина наибольшей общей подпоследовательности двух строк с восстановлением самой подпоследовательности. Для всех реализаций организовано подробное логирование промежуточных вычислений в файлы, что позволяет наглядно отслеживать заполнение таблиц динамического программирования и обосновывать корректность работы алгоритмов. Все программы успешно протестированы на различных наборах входных данных, включая граничные случаи (пустые строки, строки с полным несовпадением, строки с одинаковыми символами), результаты тестирования подтверждают корректную работу реализованных алгоритмов.

Приложение

Код программы:

main.cpp

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <fstream>
#include <iomanip>
#include <ctime>
#include <cstring>

using namespace std;

ofstream logFile;
bool logOpened = false;

void log_to_file(const string& msg) {
    if (logOpened) {
        logFile << msg;
    }
}

void print_dp_matrix(const vector<vector<int>>& dp, const string& s1,
const string& s2) {
    int m = s1.length();
    int n = s2.length();

    log_to_file("\nТаблица dp (длины НОП для префиксов):\n");
    log_to_file(" ");
    log_to_file("0 ");
    for (int j = 0; j < n; ++j) {
        log_to_file(string(1, s2[j]) + " ");
    }
    log_to_file("\n");

    for (int i = 0; i <= m; ++i) {
        if (i == 0) log_to_file("0 ");
        else log_to_file(string(1, s1[i-1]) + " ");

        for (int j = 0; j <= n; ++j) {
            if (dp[i][j] > 9) log_to_file(to_string(dp[i][j]) + " ");
            else log_to_file(to_string(dp[i][j]) + " ");
        }
        log_to_file("\n");
    }
    log_to_file("\n");
}

int main() {
    logFile.open("lcs_log.txt");
    if (!logFile.is_open()) {
        cerr << "Warning: не удалось открыть лог-файл\n";
        logOpened = false;
    } else {
        logOpened = true;
    }

    string s1, s2;
    getline(cin, s1);
```

```

getline(cin, s2);

log_to_file("Строка 1: " + s1 + "\n");
log_to_file("Строка 2: " + s2 + "\n");

int m = s1.length();
int n = s2.length();
vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

log_to_file("\nЗаполнение таблицы динамического
программирования\n");

for (int i = 1; i <= m; ++i) {
    for (int j = 1; j <= n; ++j) {
        if (s1[i-1] == s2[j-1]) {
            dp[i][j] = dp[i-1][j-1] + 1;
            log_to_file("Сравниваем '" + string(1, s1[i-1]) + "' и
'" + string(1, s2[j-1]) + "' -> равны, берём dp[" +
to_string(i-1) + "][" + to_string(j-1) + "]
+ 1 = " + to_string(dp[i][j]) + "\n");
        } else {
            dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            log_to_file("Сравниваем '" + string(1, s1[i-1]) + "' и
'" + string(1, s2[j-1]) + "' -> не равны, max(dp[" +
to_string(i-1) + "][" + to_string(j) + "] = "
+ to_string(dp[i-1][j]) + ", dp[" +
to_string(i) + "][" + to_string(j-1) + "] = "
+ to_string(dp[i][j-1]) + ") = " + to_string(dp[i][j]) + "\n");
        }
    }
    log_to_file("\n После обработки строки " + to_string(i) + "
(символ '" + s1[i-1] + "') \n");
    print_dp_matrix(dp, s1, s2);
}

log_to_file("\nТаблица полностью заполнена\n");
print_dp_matrix(dp, s1, s2);

log_to_file("\nВосстановление наибольшей общей
подпоследовательности\n");
string lcs = "";
int i = m, j = n;
while (i > 0 && j > 0) {
    log_to_file("Текущие индексы: i=" + to_string(i) + " (символ
s1[" + to_string(i-1) + "]='" + s1[i-1] + "'), j=" +
to_string(j) + " (символ s2[" + to_string(j-1) +
"]='" + s2[j-1] + "')\n");
    if (s1[i-1] == s2[j-1]) {
        log_to_file(" Символы равны: добавляем '" + string(1,
s1[i-1]) + "' в НОП и переходим по диагонали\n");
        lcs.push_back(s1[i-1]);
        --i; --j;
    } else if (dp[i-1][j] > dp[i][j-1]) {
        log_to_file(" Символы не равны, dp[" + to_string(i-1) +
"][" + to_string(j) + "] = " + to_string(dp[i-1][j]) +
" > dp[" + to_string(i) + "][" + to_string(j-1)
+ "] = " + to_string(dp[i][j-1]) + " -> идём вверх\n");
        --i;
    } else {
        log_to_file("

```

1.cpp

```
#include <iostream>
```

```

#include <vector>
#include <string>
#include <algorithm>
#include <fstream>
#include <iomanip>

bool log_flag = true;

void print_table(std::ofstream& log, const
std::vector<std::vector<int>>& dp,
const std::string& A, const std::string& B) {
    int n = A.size(), m = B.size();
    log << " ";
    for (int j = 0; j < m; ++j) log << " " << B[j];
    log << "\n";
    for (int i = 0; i <= n; ++i) {
        if (i == 0) log << " ";
        else log << A[i-1] << " ";
        for (int j = 0; j <= m; ++j)
            log << std::setw(3) << dp[i][j];
        log << "\n";
    }
    log << "\n";
}

int main() {
    int cr, ci, cd;
    std::cin >> cr >> ci >> cd;
    std::string A, B;
    std::cin >> A >> B;
    int n = A.size(), m = B.size();

    std::vector<std::vector<int>> dp(n+1, std::vector<int>(m+1));
    for (int i = 0; i <= n; ++i) dp[i][0] = i * cd;
    for (int j = 0; j <= m; ++j) dp[0][j] = j * ci;

    std::ofstream log("log1.txt");

    if (log_flag) {
        log << "стоимость_замены=" << cr << " стоимость_вставки=" <<
ci << " стоимость_удаления=" << cd << "\n";
        log << "Строка A=" << A << " Строка B=" << B << "\n\n";
        log << "Начальная таблица:\n";
        print_table(log, dp, A, B);
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            char ca = A[i-1], cb = B[j-1];
            if (ca == cb) {
                dp[i][j] = dp[i-1][j-1];
                if (log_flag) log << "Сравнение '" << ca << "' и '" <<
cb << "' -> равны, берём dp[" << i-1 << "][" << j-1 << "] = " << dp[i][j] <<
"\n";
            } else {
                int repl = dp[i-1][j-1] + cr;
                int ins = dp[i][j-1] + ci;
                int del = dp[i-1][j] + cd;
                dp[i][j] = std::min({repl, ins, del});
                if (log_flag) log << "Сравнение '" << ca << "' и '" <<
cb << "' -> не равны, min(замена=" << repl << ", вставка=" << ins << ",
удаление=" << del << ") = " << dp[i][j] << "\n";
            }
        }
    }
}

```

```

        if (log_flag) {
            log << "\nПосле строки " << i << " (символ '" << A[i-1] <<
            "):\n";
            print_table(log, dp, A, B);
        }

        if (log_flag) log << "Финальное расстояние = " << dp[n][m] << "\n";
        log.close();

        std::cout << dp[n][m] << std::endl;
        return 0;
    }

```

2.cpp

```

#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <fstream>
#include <iomanip>

bool log_flag = true;

void print_table(std::ofstream& log, const
std::vector<std::vector<int>>& dp,
                const std::string& A, const std::string& B) {
    int n = A.size(), m = B.size();
    log << " ";
    for (int j = 0; j < m; ++j) log << " " << B[j];
    log << "\n";
    for (int i = 0; i <= n; ++i) {
        if (i == 0) log << " ";
        else log << A[i - 1];
        for (int j = 0; j <= m; ++j)
            log << std::setw(3) << dp[i][j];
        log << "\n";
    }
    log << "\n";
}

int main() {
    int costR, costI, costD;
    std::cin >> costR >> costI >> costD;
    std::string A, B;
    std::cin >> A >> B;
    int n = A.size();
    int m = B.size();

    std::vector<std::vector<int>> dp(n + 1, std::vector<int>(m + 1, 0));
    for (int i = 1; i <= n; ++i) dp[i][0] = i * costD;
    for (int j = 1; j <= m; ++j) dp[0][j] = j * costI;

    std::ofstream log("log2.txt");
    if (log_flag) {
        log << "стоимость_замены=" << costR << " стоимость_вставки="
        << costI
        << " стоимость_удаления=" << costD << "\n";
        log << "Строка A=" << A << " Строка B=" << B << "\n\n";
        log << "Начальная таблица:\n";
        print_table(log, dp, A, B);
    }
}

```

```

for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        char ca = A[i - 1], cb = B[j - 1];
        if (ca == cb) {
            dp[i][j] = dp[i - 1][j - 1];
            if (log_flag) {
                log << "Сравнение '" << ca << "' и '" << cb
                    << "' -> равны, берём dp[" << i - 1 << "][" <<
j - 1
                    << "] = " << dp[i][j] << "\n";
            }
        } else {
            int repl = dp[i - 1][j - 1] + costR;
            int ins = dp[i][j - 1] + costI;
            int del = dp[i - 1][j] + costD;
            dp[i][j] = std::min({repl, ins, del});
            if (log_flag) {
                log << "Сравнение '" << ca << "' и '" << cb
                    << "' -> не равны, min(замена=" << repl
                    << ", вставка=" << ins << ", удаление=" << del
                    << ") = " << dp[i][j] << "\n";
            }
        }
    }
    if (log_flag) {
        log << "\nПосле строки " << i << " (символ '" << A[i - 1]
<< "'): \n";
        print_table(log, dp, A, B);
    }
}

if (log_flag) {
    log << "Финальное расстояние = " << dp[n][m] << "\n";
}
log.close();

std::string ops;
int i = n, j = m;
while (i > 0 || j > 0) {
    if (i > 0 && j > 0 && A[i - 1] == B[j - 1] && dp[i][j] == dp[i
- 1][j - 1]) {
        ops.push_back('M');
        i--; j--;
    } else if (i > 0 && j > 0 && dp[i][j] == dp[i - 1][j - 1] +
costR) {
        ops.push_back('R');
        i--; j--;
    } else if (j > 0 && dp[i][j] == dp[i][j - 1] + costI) {
        ops.push_back('I');
        j--;
    } else if (i > 0 && dp[i][j] == dp[i - 1][j] + costD) {
        ops.push_back('D');
        i--;
    }
}
std::reverse(ops.begin(), ops.end());

std::cout << ops << std::endl;
std::cout << A << std::endl;
std::cout << B << std::endl;

return 0;
}

```

3.cpp

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <fstream>
#include <iomanip>

using namespace std;

bool log_flag = true;

void print_table(ofstream& log, const vector<vector<int>>& dp,
               const string& A, const string& B) {
    int n = A.size(), m = B.size();
    log << "      ";
    for (int j = 0; j < m; ++j) log << " " << B[j];
    log << "\n";
    for (int i = 0; i <= n; ++i) {
        if (i == 0) log << " ";
        else log << A[i - 1];
        for (int j = 0; j <= m; ++j)
            log << setw(3) << dp[i][j];
        log << "\n";
    }
    log << "\n";
}

int main() {
    string s, t;
    cin >> s >> t;
    int n = s.size(), m = t.size();

    const int costR = 1, costI = 1, costD = 1;

    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; ++i) dp[i][0] = i * costD;
    for (int j = 1; j <= m; ++j) dp[0][j] = j * costI;

    ofstream log;
    if (log_flag) {
        log.open("log3.txt");
        log << "стоимость_замены=" << costR << " стоимость_вставки="
<< costI
        << " стоимость_удаления=" << costD << "\n";
        log << "Строка А=" << s << " Строка В=" << t << "\n\n";
        log << "Начальная таблица:\n";
        print_table(log, dp, s, t);
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            char ca = s[i - 1], cb = t[j - 1];
            if (ca == cb) {
                dp[i][j] = dp[i - 1][j - 1];
                if (log_flag) {
                    log << "Сравнение '" << ca << "' и '" << cb
<< "' -> равны, берём dp[" << i - 1 << "][" <<
j - 1
                    << "] = " << dp[i][j] << "\n";
                }
            }
        }
    }
}
```



```

    } else {
        int repl = dp[i - 1][j - 1] + costR;
        int ins  = dp[i][j - 1] + costI;
        int del  = dp[i - 1][j] + costD;
        dp[i][j] = min({repl, ins, del});
        if (log_flag) {
            log << "Сравнение '" << sa << "'" и '" << sb
                << "'" -> не равны, min(замена=" << repl
                << ", вставка=" << ins << ", удаление=" << del
                << ") = " << dp[i][j] << "\n";
        }
    }
}
}
if (log_flag) {
    log << "\nПосле строки " << i << " (символ '" << s[i - 1]
<< "'): \n";
    print_table(log, dp, s, t);
}

int distance = dp[n][m];
if (log_flag) {
    log << "Финальное расстояние = " << distance << "\n";
    log.close();
}

cout << distance << endl;

return 0;
}

```